



Lesson Objectives



- Var, Let and Const keyword
- Arrow functions, default arguments
- Template Strings, String methods
- Object de-structuring
- Spread and Rest operator
- Typescript Fundamentals
- Types & type assertions, Creating custom object types, function types
- Typescript OOPS - Classes, Interfaces, Constructor, etc
- Asynchronous Programming in ES6
- Promise Constructor
- Promise with Chain
- Promise Race



12.1: JavaScript for Test Automation

Var, Let and Const keyword



- **Var**

- Function-scoped
- Can be re-declared and updated
- Hoisted (available before declaration but with undefined)

```
function testVar() {  
  if (true) {  
    var x = 10;  
  }  
  console.log(x); // 10 (function scoped)  
}
```

12.1: JavaScript for Test Automation

Var, Let and Const keyword



- **Let**

- Block-scoped
- Cannot be re-declared in the same scope
- Can be updated

```
function testLet() {  
  if (true) {  
    let y = 20;  
  }  
  console.log(y); // Error (block scoped)  
}
```

12.1: JavaScript for Test Automation

Var, Let and Const keyword



- **Const**

- Block-scoped
- Cannot be re-declared or updated
- Value remains constant (but objects/arrays can still be mutated)

```
const z = 30;  
z = 40; // Error (cannot reassign)
```

12.2: JavaScript for Test Automation



Arrow functions, default arguments

- Arrow Functions
 - Shorter and cleaner way to write functions.
 - Do not have their own this → they use this from the surrounding scope.
 - Great for callbacks and concise code.
- Example:

```
// Normal function
function greet(name) {
  return "Hello " + name;
}

// Arrow function
const greet = (name) => "Hello " + name;
```

12.2: JavaScript for Test Automation



Arrow functions, default arguments

- Default Arguments
 - Allow parameters to have a default value if not provided.
 - Prevents undefined errors.
- Example:

```
const add = (a, b = 0) => a + b;  
  
console.log(add(5)); // 5 (since b = 0 by  
default)  
console.log(add(5, 3)); // 8
```

12.3: JavaScript for Test Automation

Template Strings & String Methods



- Template Strings (Template Literals)
 - Use backticks (` `) instead of quotes.
 - Allow embedding variables and expressions with `${ }`.
 - Support multi-line strings easily.

```
let name = "John";  
console.log(` Hello, ${name}!`); // Hello,  
John!  
  
let a = 5, b = 10;  
console.log(` Sum = ${a + b}`); // Sum = 15
```


12.3: JavaScript for Test Automation



Template Strings & String Methods

- String Methods:
 - .length → returns string length
 - .toUpperCase() → converts to uppercase
 - .toLowerCase() → converts to lowercase
 - .includes() → checks if a substring exists
 - .slice(start, end) → extracts part of string
 - .replace(old, new) → replaces part of string

```
let str = "JavaScript is fun!";  
console.log(str.length);           // 18  
console.log(str.toUpperCase());    // JAVASCRIPT IS FUN!  
console.log(str.includes("fun"));  // true
```

12.4: JavaScript for Test Automation

Object De-structuring



What is Object Destructuring?

- A way to unpack values from objects (or arrays) into variables.
- Makes code shorter and easier to read.
- Prevents repetitive object.property access.

```
const person = { name: "Sara",  
age: 25 };  
const { name, age } = person;
```

```
console.log(name); // Sara  
console.log(age); // 25
```

```
const user = { id: 101,  
username: "kunal" };  
const { username: userName  
} = user;
```

```
console.log(userName); //  
kunal
```

12.5: JavaScript for Test Automation

Spread and Rest Operators



- Spread Operator:
 - Expands (spreads) an array or object into individual elements.
 - Common uses: copying arrays/objects, merging, passing arguments.

```
let nums = [1, 2, 3];  
let copy = [...nums];  
console.log(copy); // [1, 2, 3]
```

- Rest Operator:
 - Collects multiple values into a single array.
 - Used in function parameters to handle variable arguments.

```
function sum(...args) {  
  return args.reduce((a, b) => a + b, 0);  
}  
console.log(sum(1, 2, 3, 4)); // 10
```

12.6: JavaScript for Test Automation

Typescript Fundamentals



- TypeScript is a syntactic superset of JavaScript which adds static typin
- This basically means that TypeScript adds syntax on top of JavaScript, allowing developers to add types.
- In Short:
 - Superset of JS, adds type safety.
 - File extension: .ts
 - Compiled to JavaScript.

12.7: JavaScript for Test Automation



Types & type assertions, Custom types, Function types

- Basic Types in TypeScript
 - string → "Hello"
 - number → 42
 - boolean → true / false
 - any → can hold any type (use sparingly)

```
let name: string = "Deepali";  
let age: number = 25;  
let isAdmin: boolean = false;  
let flexible: any = "Can be anything!";
```

12.7: JavaScript for Test Automation



Types & type assertions, Custom types, Function types

- **Type Assertions**

- Tell the compiler to treat a variable as a specific type.
- Useful when working with any type.

```
let val: any = "hello";  
let len: number = (val as string).length;  
console.log(len); // 5
```

12.7: JavaScript for Test Automation



Types & type assertions, Custom types, Function types

▪ Custom Object Types

- Use type keyword to define reusable structures.

```
type User = { id: number; name: string; };  
const u1: User = { id: 101, name: "Sara" };
```

▪ Function Types

- Define types for function parameters and return values.

```
function add(a: number, b: number): number {  
  return a + b;  
}  
  
type Greet = (name: string) => string;  
const greet: Greet = (name) => `Hello, ${name}`;
```

12.8: JavaScript for Test Automation

Typescript OOPS - Classes, Interfaces, Constructor, etc



Classes

- A class is a blueprint for creating objects.
- Contains properties (variables) and methods (functions).

Example:

```
class Car {  
  brand: string;  
  constructor(brand: string) {  
    this.brand = brand;  
  }  
  drive() {  
    console.log(` ${this.brand} is driving...`);  
  }  
}  
  
const c1 = new Car("Tesla");  
c1.drive(); // Tesla is driving..
```


12.8: JavaScript for Test Automation

Typescript OOPS - Classes, Interfaces, Constructor, etc



Constructor

A class is a blueprint for creating objects.

- A special method in a class that runs when an object is created.
- Used to initialize properties.

Example:

```
class Employee {  
  id: number;  
  name: string;  
  
  constructor(id: number, name: string) {  
    this.id = id;  
    this.name = name;  
  }  
}  
  
const e1 = new Employee(101, "Deepali");  
console.log(e1.name); // Deepali
```

12.8: JavaScript for Test Automation

Typescript OOPS - Classes, Interfaces, Constructor, etc



Interfaces

- Define the shape of an object (what properties/methods it should have).
- Classes can implement interfaces to enforce structure.

Example:

```
interface Person {  
  name: string;  
}  
  
class Student implements Person {  
  name: string;  
  constructor(name: string) {  
    this.name = name;  
  }  
}
```

12.9: JavaScript for Test Automation

Asynchronous Programming in ES6



Why Asynchronous?

- JavaScript is single-threaded (one task at a time).
- Asynchronous programming allows tasks like API calls, file reads, or timers to run without blocking the main program.

Promises

- A Promise represents a value that may be available now, later, or never.
- States:
 - Pending → Initial state
 - Resolved (Fulfilled) → Success
 - Rejected → Error

12.9: JavaScript for Test Automation



Asynchronous Programming in ES6

```
let promise = new Promise((resolve, reject) => {  
  setTimeout(() => resolve("Data received!"), 2000);  
});  
  
promise.then(result => console.log(result))  
  .catch(err => console.log(err));
```

Async/Await

- `async` makes a function return a Promise.
- `await` pauses execution until the Promise is resolved.
- Cleaner and easier to read than `.then()` chains.

```
async function fetchData() {  
  let result = await Promise.resolve("Data loaded!");  
  console.log(result);  
}  
fetchData(); // Data loaded!
```

12.10: JavaScript for Test Automation

Promise Constructor



- The Promise constructor in JavaScript is used to create a new Promise object. It is designed to handle asynchronous operations and manage their eventual success (fulfillment) or failure (rejection).

```
let promise = new Promise((resolve, reject) => {  
  resolve("Success!");  
});
```

12.11: JavaScript for Test Automation

Promise with Chain



- Promise chaining, in JavaScript, is a way to manage multiple asynchronous operations sequentially.
- It allows you to execute operations one after another, where the output of one promise becomes the input for the next.

```
doTask()  
  .then(result => process(result))  
  .then(final => console.log(final));
```

12.12: JavaScript for Test Automation

Promise Race



- `Promise.race()` is a static method in JavaScript that takes an iterable (like an array) of Promises as input and returns a single Promise.
- This returned Promise settles with the eventual state (either fulfilled or rejected) of the first Promise in the input iterable that settles.

```
Promise.race([p1, p2, p3]).then(result =>  
  console.log(result));
```

Review Question



Question 1: Which of the following is block-scoped?

- a) var
- b) let
- c) const
- d) Both b & c

Question 2: What will this code print?

```
const nums = [1,2,3];  
console.log([...nums, 4]);
```

- a) [1,2,3]
- b) [1,2,3,4]
- c) 4,3,2,1
- d) Error



Question 1 – Option D

Question 2 – Option B

Summary



In this lesson, you have learnt

- Var, Let and Const keyword
- Arrow functions, default arguments
- Template Strings, String methods
- Object de-structuring
- Spread and Rest operator
- Typescript Fundamentals
- Types & type assertions, Creating custom object types, function types
- Typescript OOPS - Classes, Interfaces, Constructor, etc
- Asynchronous Programming in ES6
- Promise Constructor
- Promise with Chain
- Promise Race



Add the notes here.